

EULER'S PHI FUNCTION – $\phi(n)$ = count of $x \leq n$ coprime to n

```
int phi(int n) {
    int result = n;
    for (int p=2; (long long)p*p<=n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            result -= result / p;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}
```

LCS – Longest Common Subsequence: $dp[i][j]$ = LCS of $a[0..i)$, $b[0..j)$

```
// match:  $dp[i][j]=dp[i-1][j-1]+1$  | no match:  $dp[i][j]=\max(dp[i-1][j], dp[i][j-1])$ 
vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
for (int i=1; i<=n; i++) for (int j=1; j<=m; j++)
     $dp[i][j] = (a[i-1]==b[j-1]) ? dp[i-1][j-1]+1 : \max(dp[i-1][j], dp[i][j-1]);$ 
// answer =  $dp[n][m]$ 
```

LIS – Longest Increasing Subsequence, $O(n \log n)$ via lower bound

```
vector<int> tails;
for (int x : a) {
    auto it = lower_bound(tails.begin(), tails.end(), x); // use upper_bound for
    // non-decreasing
    if (it == tails.end()) tails.push_back(x); else *it = x;
}
// answer = tails.size() (tails itself is NOT the actual LIS, only its length)
```

EDIT DISTANCE – min ops (insert/delete/replace) to turn a into b

```
vector<vector<int>> dp(n+1, vector<int>(m+1));
for (int i=0; i<=n; i++)  $dp[i][0] = i;$ 
for (int j=0; j<=m; j++)  $dp[0][j] = j;$ 
for (int i=1; i<=n; i++) for (int j=1; j<=m; j++) {
    if ( $a[i-1]==b[j-1]$ )  $dp[i][j] = dp[i-1][j-1];$ 
    else  $dp[i][j] = 1 + \min(\{dp[i-1][j-1], dp[i-1][j], dp[i][j-1]\});$  // replace,
    // delete, insert
}
// answer =  $dp[n][m]$ 
```

INTERVAL & SCHEDULING GREEDY – sort by end time, pick greedily

```
sort(intervals.begin(), intervals.end(), [](auto&a, auto&b){ return a.second<b.second;
});
int count = 0; long long lastEnd = LLONG_MIN;
for (auto& [s, e] : intervals) {
    if (s >= lastEnd) { count++; lastEnd = e; } // non-overlapping, greedily extend
}
// count = max number of non-overlapping intervals selectable
// key note: sorting by END time (not start) is what makes this greedy provably
// optimal
```

// $\phi(n)=n \cdot \prod (1-1/p)$ over distinct prime factors

// LIS DP: $dp[i]=1+\max(dp[j])$ ($a[j]<a[i]$)

// lower_bound -> strict LIS

// upper_bound -> non-decreasing LIS

// LCS/Edit Distance -> backtrack from $dp[n][m]$ to recover answer